

TaskForge

Gestion de tickets d'incidents — helpdesk interne

Hackathon interne ForgeWorks · MVP livré en 2 semaines

Équipe : Arthur, Abd-Ellah, Ibrahima

Soutenance du 07/08/2026

En un coup d'œil

React 19 · NestJS 11 · PostgreSQL 16

94 story points livrés sur 96

106 tests unitaires, 11 suites

7 ADR datées et justifiées

CI verte sur main et develop

Images prod : -38 % et -85 %

Le produit

Déclarer un incident, le router vers un technicien, le suivre jusqu'à sa résolution.

Utilisateur

Crée un ticket : titre,
description, priorité

Suit l'avancement de ses
déclarations

Technicien

Voit les tickets qui lui
sont assignés

Fait avancer les statuts
jusqu'à la résolution

Admin

Gère les comptes et les
rôles

Assigne et réassigne, voit
l'ensemble du parc

Couverture du cahier des charges

- CRUD complet — priorité (basse → critique), statut (ouvert → en cours → résolu → fermé)
- Assignment et réassignment d'un ticket à un autre technicien
- Filtrage par statut, priorité et technicien · tri · recherche textuelle titre + description
- Dashboard : tickets ouverts / en cours / résolus, temps moyen de résolution, répartition

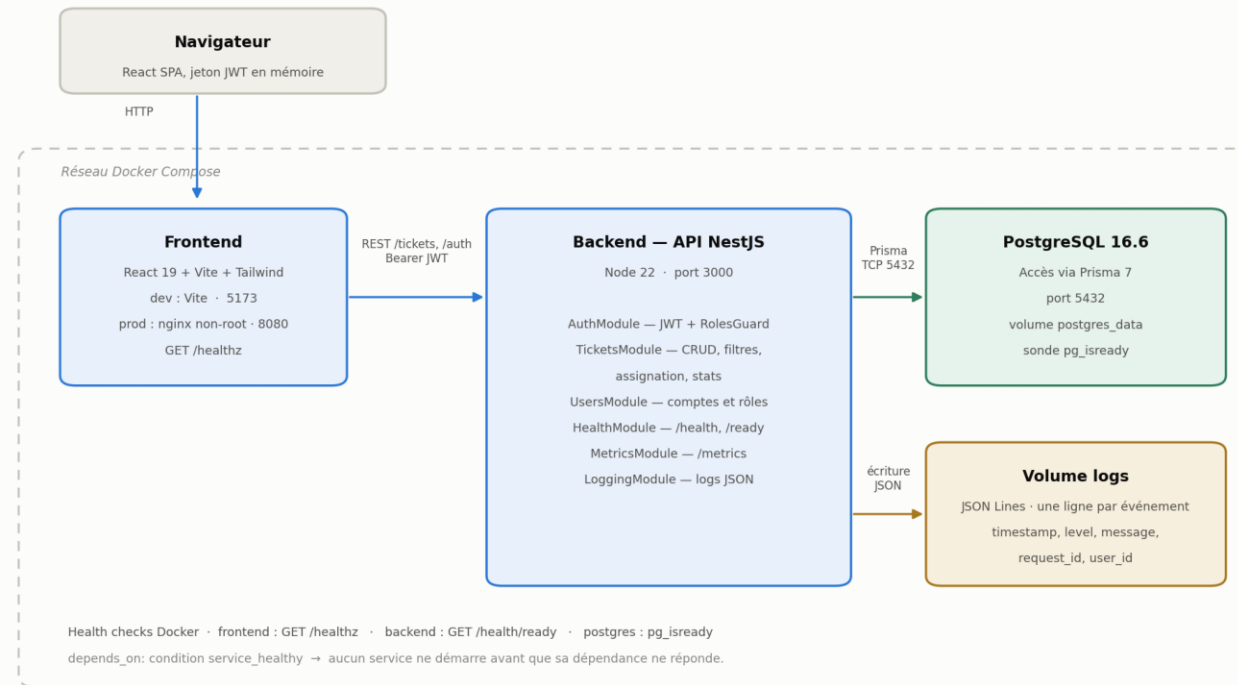
Hors périmètre assumé : notifications temps réel, commentaires et export — classés Could au MoSCoW, non engagés au sprint.

Architecture

Trois services conteneurisés, un seul langage de bout en bout.

TaskForge — diagramme de composants

Services, flux de données et sondes de santé · généré par docs/generate_architecture.py



Pourquoi ce choix

TypeScript sur les trois couches : un seul contexte mental, types partagés.

NestJS impose sa structure — modules, DI, gardes. On n'invente pas d'architecture quand le temps est la contrainte principale.

Prisma : schéma déclaratif qui documente la BDD.

Monitoring d'abord écrit à la main, puis passé sur pino et prom-client (ADR-008) : le choix maison tenait tant que le besoin s'arrêtait à l'exposition, plus dès qu'on vise un vrai collecteur.

Décisions structurantes

Sept ADR écrites au moment de la décision, pas reconstituées à la fin.

ADR	Décision	Statut
001	Stack React / NestJS / PostgreSQL / Prisma	Acceptée
002	Séparation dev / prod : root en dev, non-root en prod	Acceptée
003	Développement solo assumé	Remplacée par 007
004	Schéma : deux relations distinctes vers User	Acceptée
005	Prisma 7 dans NestJS : CJS, driver adapter	Acceptée
006	JWT Bearer + RolesGuard sur /tickets	Acceptée
007	Contribution partielle constatée	Acceptée

ADR-003 → ADR-007 : une décision trop rapide, corrigée

Le 31/07, après deux jours d'observation dont un vendredi, ADR-003 actait un développement solo.
Le 03/08, Abd-Ellah livrait 24 points. ADR-003 n'est pas réécrite : elle reste au dossier, marquée remplacée. Effacer une erreur d'appréciation, c'est perdre la matière de la rétrospective.

Observabilité

Santé, logs et métriques — pino et prom-client, contrat de sortie inchangé.

Endpoints

Route	Rôle
GET /health	État détaillé, 503 si dégradé
GET /health/live	Le processus répond
GET /health/ready	503 tant que la base est muette
GET /metrics	Format Prometheus 0.0.4
GET /healthz	Frontend, nginx en prod

Métriques exposées

taskforge_tickets_created_total · taskforge_http_requests_total{method,status}
taskforge_http_request_duration_seconds (histogramme) · taskforge_users_connected

Plus 73 séries process et Node fournies par collectDefaultMetrics() : CPU, mémoire, event loop.

Logs JSON structurés — pino

Une ligne par événement sur stdout,
dupliquée dans le volume Docker
partagé taskforge_logs.

timestamp · level · message
request_id · user_id

request_id propagé par
AsyncLocalStorage et renvoyé au
client dans x-request-id.

Une honnêteté technique : Docker Compose ne redémarre pas un conteneur unhealthy — seul Swarm le fait.
restart: unless-stopped couvre le crash, depends_on: service_healthy couvre l'ordre de démarrage.

Qualité et tests

106 tests unitaires sur 11 suites, verts à chaque push.

Logique métier testée

Transitions de statut :
ouvert → en cours → résolu
et les transitions interdites

Calcul du temps moyen
de résolution

Avant chaque commit

ESLint + Prettier

Hook pre-commit Husky +
lint-staged : un commit non
conforme est bloqué,
pas seulement signalé.

CI GitHub Actions

Deux jobs à chaque push
et chaque PR sur main
et develop :

backend — lint, build, test
frontend — lint, build

La discipline qui va avec

- Aucun commit direct sur develop : branche, PR, CI verte, puis merge
- isValidStatusTransition() isolée en fonction pure — testable sans Prisma, pensée pour ça dès ADR-005

Aucune régression n'a atteint la branche d'intégration sur les deux sprints.

Docker

Multi-stage, séparation dev / prod, durcissement là où il a du sens.

Gain des multi-stage builds

Image	Dev	Prod	Gain
taskforge-backend	1.28 Go	792 Mo	≈ 38 %
taskforge-frontend	521 Mo	76.4 Mo	≈ 85 %

Sécurité appliquée

Conteneurs non-root : USER app / USER nginx
.dockerignore exclut .env, node_modules, tests
Aucun secret dans l'image — injection runtime
Tags versionnés via IMAGE_TAG, jamais :latest

Deux configurations

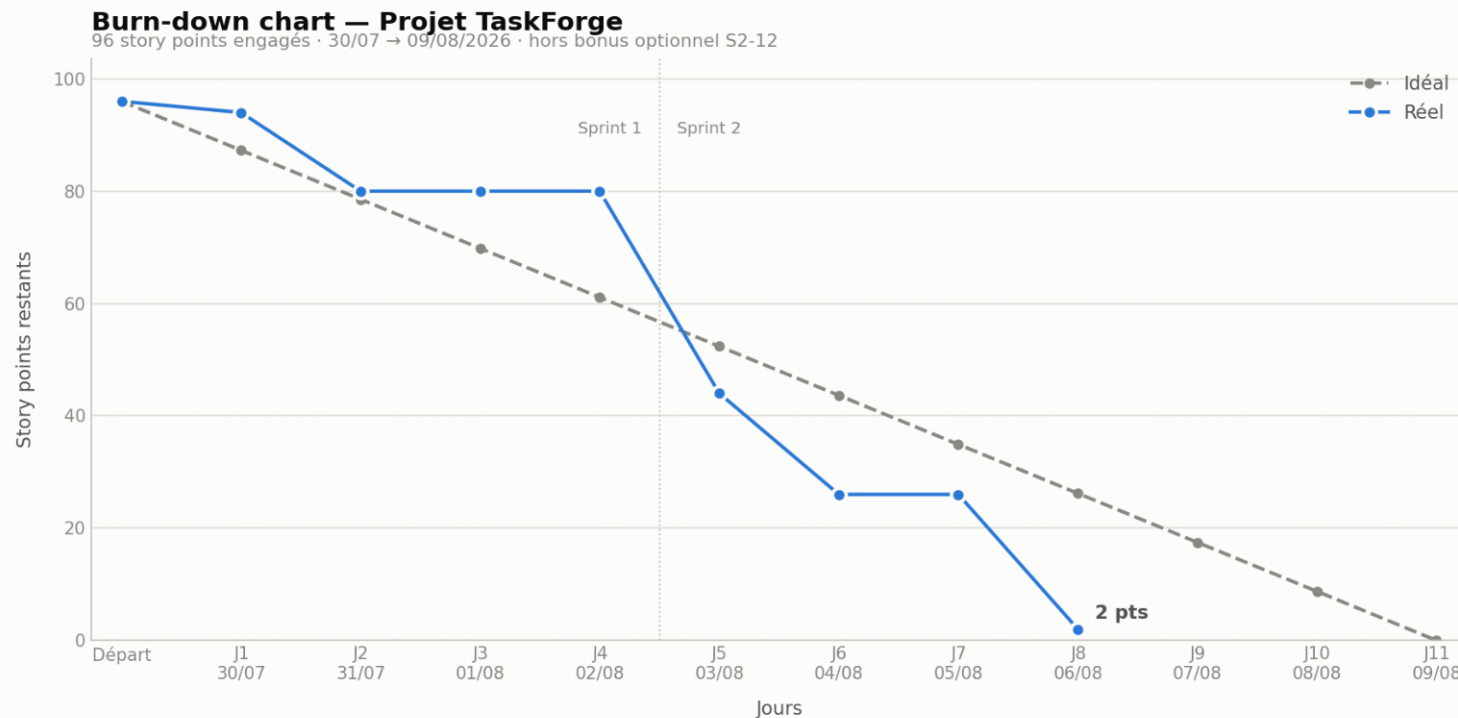
docker-compose.yml — volumes montés,
hot-reload Vite et Nest, debug actif

docker-compose.prod.yml — images buildées,
pas de source montée, env de production

Root en dev, non-root en prod : forcer USER node sur les images de dev provoque un EACCES au démarrage de Vite (volume anonyme node_modules appartenant à root). L'exigence est appliquée là où elle protège.

Gestion de sprint

Deux sprints d'une semaine, 96 points engagés hors bonus.



Sprint goals

S1 — une application fonctionnelle de bout en bout sous Docker, CI en place.

S2 — monitoring, Docker durci, tests, dashboard et filtres.

Artefacts tenus

Kanban GitHub Projects
Backlog estimé en Fibonacci
9 daily logs
Burn-down idéal vs réel
Rétrospective Keep/Drop/Try

Vélocité : le nominal contre le réel

L'écart entre ce qui était planifié et ce qui a été engagé est le fait marquant du projet.

Points engagés, par personne

	Assigné au backlog	Réellement engagé
Arthur	40 pts	70 pts
Abd-Allah	30 pts	24 pts
Ibrahima	24 pts	0 pt

Vélocité mesurée

Sprint 1 clos : 16 points livrés sur 52 engagés — 31 %. C'est la seule valeur réellement close, et donc la seule base honnête pour estimer un sprint suivant. Estimer 52 points sur 4 jours pour « trois personnes » supposait 13 points par jour et par personne — un chiffre jamais confronté à une capacité observée.

La moyenne ment

Trois journées concentrent
78 des 94 points livrés,
soit 83 %.

La moyenne de 11,75 pts/jour
ne décrit aucune journée
réelle du projet : le burn-down
suit le rythme de merge, pas
la production.

Périmètre final : 94 points livrés sur 96 engagés. Reste le screencast (S2-11, 2 pts).

L'enseignement du sprint

24 points assignés à quelqu'un qui n'avait pas les droits de push.

« N'a pas contribué » et « n'a pas pu contribuer » n'appellent pas la même réponse.

Ibrahima n'a jamais été collaborateur du dépôt. Ce n'est pas un problème de motivation mais d'onboarding — et l'équipe ne s'en est aperçue que le 05/08, à quatre jours du rendu. Le premier cas appelle une redistribution des tâches, le second l'ouverture d'un accès. Traiter le second comme le premier, c'est redistribuer du travail sans corriger la cause.

Le coût réel

Aucune rotation des rôles possible :
elle suppose deux contributeurs
disponibles avec les mêmes droits.

Arthur a tenu PM et dev principal du
30/07 au 06/08, sans regard extérieur.

Ce qu'on change au prochain sprint

Vérifier les accès avant d'assigner —
contrôle d'ouverture de sprint, 1 minute.

Estimer sur la vélocité mesurée (16 pts),
pas sur l'effectif déclaré, et fermer
l'issue depuis la PR : Closes #N.

Symptôme du même angle mort : daily logs, rétrospective et support de soutenance n'avaient aucun ticket au backlog jusqu'au 05/08.

Rétrospective

Keep / Drop / Try — retour critique sur la gestion du sprint.

Keep

Estimer et découper dès J1 :
le backlog chiffré existait
avant la première ligne de code.

Écrire les ADR au moment
de la décision.

Branche + PR + CI verte,
sans exception.

Le lot Docker/CI livré d'un
bloc : le seul périmètre
réellement délégué.

Drop

Estimer sur l'effectif déclaré
plutôt qu'observé.

Le board comme vitrine : cinq
tickets livrés sont restés
ouverts jusqu'au 05/08.

L'assignation nominative
non vérifiée.

Le rattrapage des artefacts
en fin de course.

Try

Vérifier les accès avant
d'assigner.

Estimer sur la vélocité
mesurée : base 16 points.

Fermer l'issue depuis la PR
— le board se met à jour
seul au merge.

Un ticket par livrable du
cahier des charges,
artefacts PM inclus.

Ces artefacts ont documenté le projet au lieu de le piloter : un burn-down à jour au 02/08 rendait le décrochage visible quatre jours plus tôt.

Ce qui est livré

Le dépôt reflète l'état réel du projet : main est à jour.

Livrable	État	Où
Application complète (front + back + BDD)	Livré	docker compose up
Conteneurisation dev et prod	Livré	docker-compose*.yaml
Schéma d'architecture + 7 ADR	Livré	docs/
Kanban, backlog estimé, 9 daily logs	Livré	pm/ + GitHub Projects
Burn-down idéal vs réel	Livré	pm/burndown.png
Rétrospective Keep / Drop / Try	Livré	pm/retrospective.md
Screencast de démo	En cours	docs/screencast.mp4

Deux chiffres pour finir

94 points livrés sur 96 engagés. Et 16 points — la vélocité réellement mesurée sur le seul sprint clos, celle sur laquelle on estimera le prochain.

Démonstration en direct — création, assignation, résolution d'un ticket.